

Team Project 1 - CAP

JavaScript Fundamental

Amjad



Chapter 1
Introduction to Javascript

Anjad

What is javascript?

- JavaScript (JS) is a high level, dynamic programming language widely used in web development.
- It is one of the three core web technologies:
  - HTML: Defines the structure and content of web pages.
  - CSS: Controls the appearance and styling.
  - JavaScript: Adds behavior, logic, and interactivity.
- JavaScript was created by Brendan Eich in 1995.
- JavaScript and Java are completely different programming languages.

Is JavaScript a Programming, Scripting, or Markup Language?

Type	Description	Example
Markup Language	Defines the structure of content	HTML
Programming Language	Provides instructions, logic, and algorithms	JavaScript, Java, Python
Scripting Language	A programming language commonly used to automate tasks and control software behavi or	JavaScript, Python

JavaScript is a programming language that is also commonly classified as a scripting language. It is **not** a markup language.



How Does JavaScript Execute?

Computers execute machine code, so programming languages need a way to translate or process their instructions.

Two common approaches are:

- **Interpreter**: Executes a program through a runtime environment, often processing instructions as the program runs.
- **Compiler**: Translates source code into another form, often machine code, before execution.

How JavaScript Works:

Modern JavaScript engines, such as Google's V8, use both interpretation and Just-In-Time (JIT) compilation to execute JavaScript efficiently.

- JavaScript is traditionally described as an interpreted language.
- Modern JavaScript engines also use compilation internally to improve performance.
- JavaScript runs inside web browsers and in other environments, such as Node.js.
- Interpreted and compiled are execution approaches, not mutually exclusive categories of programming languages.

Amjad

JavaScript in HTML

- JavaScript is the default scripting language in HTML.
- JavaScript was originally created to make web pages interactive and dynamic.
- JavaScript programs are called scripts.
- Scripts can be written directly inside HTML using the <script> element or loaded from external .js files.
- Scripts are executed by the browser's JavaScript engine.
- JavaScript does not require a separate manual compilation step to run in modern browsers.

```html
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Example</title>
</head>
<body>
    <h1 id="message">Hello!</h1>

    <script>
        document.getElementById("message").innerHTML = "Hello JavaScript!";
    </script>
</body>
</html>
```

Result: The heading changes from "Hello!" to "Hello JavaScript!" when the script executes.

What Can JavaScript Do?

JavaScript can dynamically control web pages and interact with users.

Capability	Description
Modify HTML Content	Add, remove, or update elements and text using the DOM
Change Attributes	Modify attributes such as src, href, and class
Change CSS Styles	Dynamically update colors, fonts, visibility, and layouts
Handle User Events	Respond to clicks, mouse movements, and keyboard input
Communicate with Servers	Send and receive data using Fetch API or AJAX techniques
Store Data	Save information using cookies and browser storage
Interact with Users	Display messages, alerts, and confirmation dialogs

```
// Change HTML content
document.getElementById("demo").innerHTML = "Welcome!";

// Change CSS style
document.getElementById("demo").style.color = "blue";

// Change an attribute
document.getElementById("photo").src = "newphoto.jpg";

// Display a message
alert("Hello JavaScript!");
```



What Makes JavaScript Unique?

- **HTML/CSS Integration**: JavaScript works directly with HTML elements and CSS styles.
- **Easy to Get Started**: Simple tasks can be implemented with a few lines of code.
- **Browser Support**: Supported by all major modern web browsers and generally enabled by default.
- **Immediate Execution**: JavaScript can run directly in the browser without a separate manual compilation process.
- **Event-Driven**: JavaScript can respond to user interactions and browser events.
- **Versatility**: JavaScript can be used for front-end, back-end, mobile, and desktop applications.

Amjad



Adding JavaScript to HTML

JavaScript can be integrated into HTML pages using two approaches:

1. **Internal JavaScript**: Write JavaScript code directly inside the HTML document using the <script> element.
2. **External JavaScript**: Write JavaScript code in a separate .js file and link it to the HTML document.

```
<!DOCTYPE html>
<html>
<body>
    <h2 id="demo">Hello!</h2>

    <script>
        document.getElementById("demo").innerHTML = "Hello JavaScript!";
    </script>
</body>
</html>
```

```
<h2 id="demo">Hello!</h2>
<script src="script.js"></script>
```

Where to Place JavaScript in HTML?

Option 1: Inside <head>

```
<!DOCTYPE html>
<html>
<head>
    <script src="script.js" defer></script>
</head>
<body>
    <h1>My Website</h1>
</body>
</html>
```

Using defer allows the browser to download th e script without blocking HTML parsing and exe cute it after the document has been parsed.

Option 2: At the End of <body>

```
<!DOCTYPE html>
<html>
<head>
    <title>My Website</title>
</head>
<body>
    <h1>My Website</h1>

    <script src="script.js"></script>
</body>
</html>
```

Placing the script at the end of <body> allows t he browser to process the page content before executing the script.

Inside <head> with defer	End of <body>
Script downloads while HTML is being parsed	Script is encountered after preceding HTML
Executes after HTML parsing finishes	Executes when encountered
Keeps script references organized	Simple and beginner-friendly
Recommended for many modern applications	Also a valid approach

Amjad

External JavaScript Files

External JavaScript is written in a separate file with the .js extension and linked to HTML using the src attribute of the <script> element.

```
<script src="script.js"></script>
```

Advantages of External JavaScript

- **Separation of concerns**: Keeps HTML structure separate from JavaScript logic.
- **Reusability**: The same JavaScript file can be used across multiple HTML pages.
- **Maintainability**: Makes code easier to read, edit, and debug.
- **Performance**: External scripts can be cached by browsers, reducing repeated downloads.

Amjad

JavaScript Comments

Comments are notes written inside source code that are ignored during JavaScript execution.

Why Use Comments?

- Explain the purpose and logic of code.
- Improve code readability and maintainability.
- Help other developers understand the code.
- Document important decisions or instructions.

Types of JavaScript Comments

1. Single-Line Comments (//)

Used to comment on a single line or part of a line.

```
// This is a single-line comment
let age = 25; // Store the user's age
```

2. Multi-Line Comments (/* ... */)

Used to write comments spanning multiple lines.

```
/*
This is a multi-line comment.
It can contain several lines.
*/
let name = "John";
```

Multi-line comments can also appear within a line:

```
let total = 10 /* first value */ + 20;
```

Amjad

JavaScript Output

JavaScript provides several ways to display information to users or developers.

Common JavaScript Output Methods:

Method	Description
alert()	Displays a popup message in the browser
console.log()	Prints information to the browser's developer console
document.write()	Writes content directly into the HTML document
innerHTML	Changes the HTML content inside an element

Examples

1. Using alert()

```
alert("Hello World!");
```

Displays a popup message with an OK button.

2. Using console.log()

```
console.log("Hello World!");
```

Displays a message in the browser's developer console.

3. Using document.write()

```
document.write("Hello World!");
```

Writes text into the HTML document.

4. Using innerHTML

```
document.getElementById("demo").innerHTML = "Hello World!";
```

Changes the content of an HTML element with id="demo".

Important Notes:

- alert() blocks interaction with the page until the user closes the dialog.
- console.log() is particularly useful for learning and debugging.
- document.write() is generally discouraged in modern web development, especially after a page has loaded, because it can replace the existing document.
- innerHTML should not be used with untrusted input without appropriate sanitization. Use textContent for plain text.

Amjad



Chapter 2
Variables & Data Types

Amjad



JavaScript Variables

What Is a Variable?

- A variable is a named container used to store data values in a program.
- Variables allow us to store, access, and manipulate data.
- A variable has a name and holds a value.
- Variables can store different types of data, such as numbers, strings, and booleans.
- JavaScript is dynamically typed, meaning variables are not restricted to a single data type.

```
let name = "John";
let age = 25;
let isStudent = true;

console.log(name);      // John
console.log(age);       // 25
console.log(isStudent); // true
```

Declaring Variables in JavaScript

JavaScript provides three keywords for declaring variables:

- `var` : The traditional way to declare variables (before ES6).
- `let` : Declares a variable whose value can be reassigned.
- `const` : Declares a variable whose binding cannot be reassigned.

Feature	var	let	const
Reassignment	Yes	Yes	No
Redeclaration in same scope	Yes	No	No
Scope	Function	Block	Block
Must initialize at declaration	No	No	Yes

```
// Using var
var city = "London";
city = "Paris"; // Allowed

// Using let
let age = 20;
age = 21; // Allowed

// Using const
const PI = 3.14159;
PI = 3.14; // TypeError
```

Best Practices

- Use const by default for values that do not need reassignment.
- Use let when the value needs to change.
- Avoid `var` in modern JavaScript code.

Anjad

JavaScript Variable Naming Rules

- Names can contain letters, digits, underscores (_), and dollar signs ($).
- Names cannot start with a digit.
- Names are case-sensitive (age and Age are different).
- Names cannot be reserved JavaScript keywords, such as let, class, or return.
- Spaces and hyphens are not allowed in variable names.

Naming Best Practices

- Use descriptive names: studentAge instead of x.
- Use camelCase for variables containing multiple words: firstName, totalPrice.
- Prefer meaningful English names for consistency.
- Avoid unnecessary abbreviations.

```
// Valid Names:
let firstName = "John";
let age2 = 25;
let _total = 100;
let $price = 50;
let studentAge = 20;

// Invalid Names:
let 2age = 25;
let first-name = "John";
let first name = "John";
let class = "A";
```

Anjad

var vs let

In JavaScript, both var and let are used to declare variables, but they have major differences in **scope**, **hoisting**, and **re-declaration**.

1. Scope (Where the variable is accessible)

- **var (Function-scoped):** If a var variable is declared inside a function, it is only accessible within that function. However, if it is declared inside a block (like an if statement or a for loop), it **leaks** out of that block and is accessible throughout the entire function or global scope.
- **let (Block-scoped):** A let variable is confined strictly to the block {} in which it is declared. This prevents variables from accidentally leaking out of loops or conditional statements.

```
function example() {
  if (true) {
    var x = "I am var";
    let y = "I am let";
  }
  console.log(x); // Works fine: "I am var"
  console.log(y); // ReferenceError: y is not defined
}
```

2. Hoisting (How the interpreter handles declarations)

- **var:** Variables declared with var are hoisted to the top of their enclosing scope and are automatically initialized with a value of undefined. This means you can access a var variable before it is declared in your code without getting an error (though its value will be undefined until the assignment line).
- **let:** Variables declared with let are also hoisted to the top of their block, but **they are not initialized**. Accessing a let variable before its declaration results in a ReferenceError. The time between entering the block and the actual declaration is known as the **Temporal Dead Zone (TDZ)**.

```
console.log(a); // Output: undefined (hoisted)
var a = 5;

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 10;
```

var vs let

In JavaScript, both var and let are used to declare variables, but they have major differences in **scope**, **hoisting**, and **re-declaration**.

3. Re-declaration in the same scope

- **var:** Allows you to re-declare the same variable multiple times within the same scope without any errors. This can easily lead to bugs where a variable is accidentally overwritten.
- **let:** Throws a SyntaxError if you try to re-declare the same variable within the same scope.

```
var count = 1;
var count = 2; // Perfectly legal with var

let name = "Alice";
let name = "Bob"; // SyntaxError: Identifier 'name' has already been declared
```

Summary Table

Feature	var	let
Scope	Function-scoped	Block-scoped
Hoisted?	Yes (initialized as undefined)	Yes (not initialized / Temporal Dead Zone)
Re-declaration	Allowed	Not allowed in the same scope
Global Object Property	Creates a property on the global window object	Does **not** create a property on the global object

Which one should you use?
In modern JavaScript (ES6 and later), **you should almost always use let** (or const if the variable won't be reassigned).
var is considered outdated and is generally only encountered in legacy codebases.

JavaScript Data Types

What Are Data Types?

A data type defines the kind of value stored in a variable and determines which operations can be performed on that value.

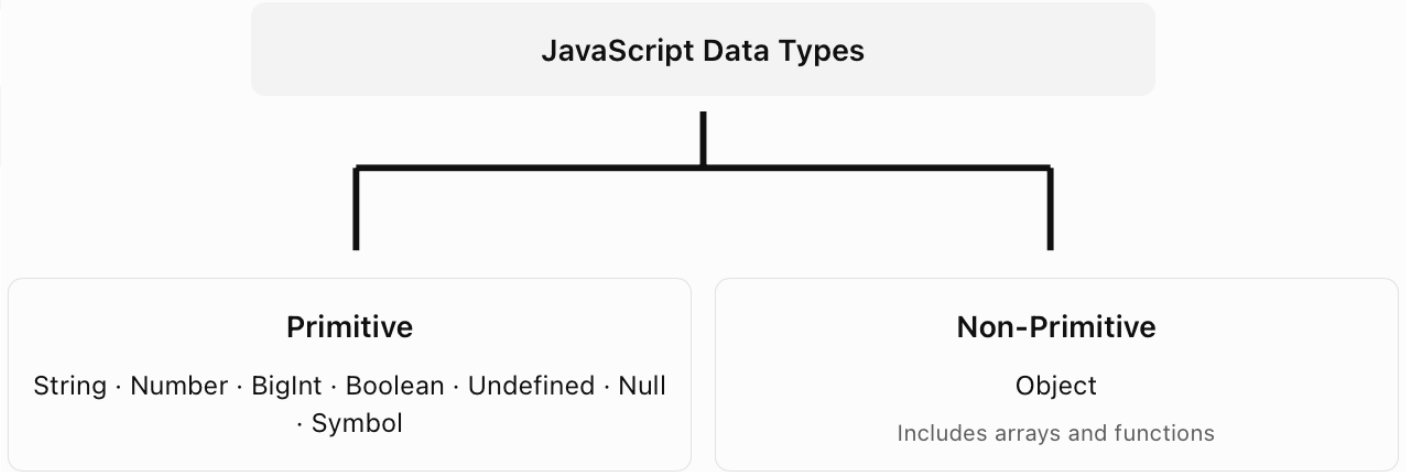
- JavaScript is a dynamically typed language.
- You do not need to specify a variable's data type when declaring it.
- JavaScript determines the type of a value at runtime.
- A variable can hold values of different types during program execution.

```
let name = "John";          // String
let age = 25;               // Number
let isStudent = true;       // Boolean
let address;                // Undefined
let salary = null;          // Null
```

JavaScript Data Types

JavaScript has 8 fundamental data types, divided into two categories:

Primitive Data Types	Non-Primitive Data Type
String	Object
Number	
BigInt	
Boolean	
Undefined	
Null	
Symbol	



JavaScript Primitive Data Types

Primitive data types represent basic values in JavaScript.

Data Type	Description	Example
String	Text enclosed in quotes	"Hello"
Number	Integers and decimal numbers	25, 3.14
BigInt	Integers of arbitrary precision	123456789n
Boolean	Logical values	true, false
Undefined	A value that has not been assigned	let x;
Null	Intentional absence of a value	let x = null;
Symbol	A unique identifier value	Symbol("id")

```
let studentName = "Sara";
let studentAge = 22;
let average = 89.5;
let graduated = false;
let phone;
let address = null;
```

Important Notes

- Strings can use single quotes ('Hello'), double quotes ("Hello"), or backticks (`Hello`).
- Both integers and decimals use the Number type.
- undefined and null are different:
- undefined: A value has not been assigned.
- null: A value has intentionally been set to represent no value.

Anjad

JavaScript Objects and the `typeof` Operator

Non-Primitive Data Types

- An object is a collection of properties and values. JavaScript objects can represent more complex data structures.
- Arrays and functions are also objects in JavaScript, although functions have a special `typeof` result.

```
// Object
let student = {
    name: "John",
    age: 25
};

// Array
let colors = ["Red", "Green", "Blue"];

// Function
function greet() {
    return "Hello!";
}
```

The typeof Operator

The `typeof` operator returns a string describing the type of a value.

```
console.log(typeof "Hello");   // "string"
console.log(typeof 25);        // "number"
console.log(typeof true);      // "boolean"
console.log(typeof undefined);// "undefined"
console.log(typeof {});        // "object"
console.log(typeof []);        // "object"
console.log(typeof greet);     // "function"

console.log(typeof null); // "object"
```



Chapter 3
Comparison & if statement

Amjad



Comparison Operators

What Are Comparison Operators?

Comparison operators are used to compare two values. The result of a comparison is always a Boolean value: true or false.

Operator	Description	Example	Result
==	Equal to (value, with type conversion)	5 == "5"	true
===	Strict equal to (no type conversion)	5 === "5"	false
!=	Not equal to (with type conversion)	5 != "5"	false
!==	Strict not equal to	5 !== "5"	true
>	Greater than	10 > 5	true
<	Less than	10 < 5	false
>=	Greater than or equal to	10 >= 10	true
<=	Less than or equal to	5 <= 3	false

```javascript
let age = 20;

console.log(age >= 18);    // true
console.log(age < 18);     // false
console.log(age == "20"); // true
console.log(age === "20");// false
```

Important: `==` vs. `===`

`==` Loose Equality

Allows type conversion

JavaScript

```javascript
5 == "5"
// true
```

Values match after conversion

`===` Strict Equality

Does not convert types

JavaScript

```javascript
5 === "5"
// false
```

Number ≠ String

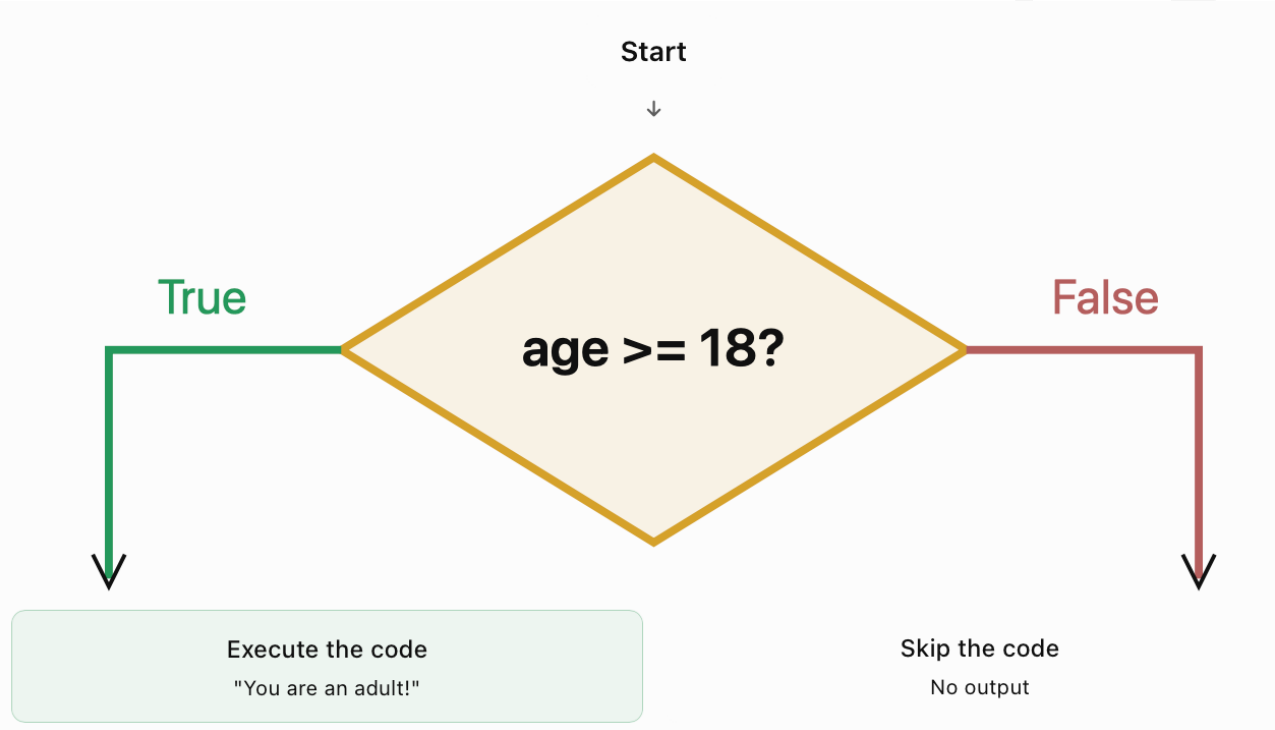
Best practice: Prefer `===` and `!==` over `==` and `!=` to avoid unexpected results caused by automatic type conversion.

The if Statement

What Is an if Statement?

The if statement is used to make decisions in JavaScript. It executes a block of code only if a specified condition is true.

- The condition is evaluated as true or false.
- If the condition is true, the code inside { } executes.
- If the condition is false, the code block is skipped.




```
let age = 20;

if (age >= 18) {
    console.log("You are an adult!");
} else {
    console.log("You are a child!");
}

// Result: You are an adult!
```



Chapter 4
Loops

Amjad



Javascript Loops

What Are Loops?

Loops are used to execute a block of code repeatedly as long as a specified condition is satisfied.

- Reduce code repetition.
- Automate repetitive tasks.
- Execute statements multiple times.
- Control repetition using conditions.

Types of JavaScript Loops

Loop	Description	Best Used When
for	Repeats code using initialization, condition, and update	Number of iterations is known
while	Repeats code while a condition is true	Number of iterations is unknown
do...while	Executes code once, then repeats while the condition is true	Code must execute at least once

Amjad

JavaScript for Loop

What Is a for Loop?

The for loop is used to repeat a block of code, typically when the number of iterations is **known**.

- Initialization: Runs once before the loop starts.
- Condition: Checked before each iteration. The loop continues while it is true.
- Update: Executes after each iteration.

```
for (initialization; condition; update) {
    // Code to execute
}
```

```
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

How the loop executes

Iteration 1	Iteration 2	Iteration 3	Iteration 4	Iteration 5
1	2	3	4	5
i = 1	i = 2	i = 3	i = 4	i = 5

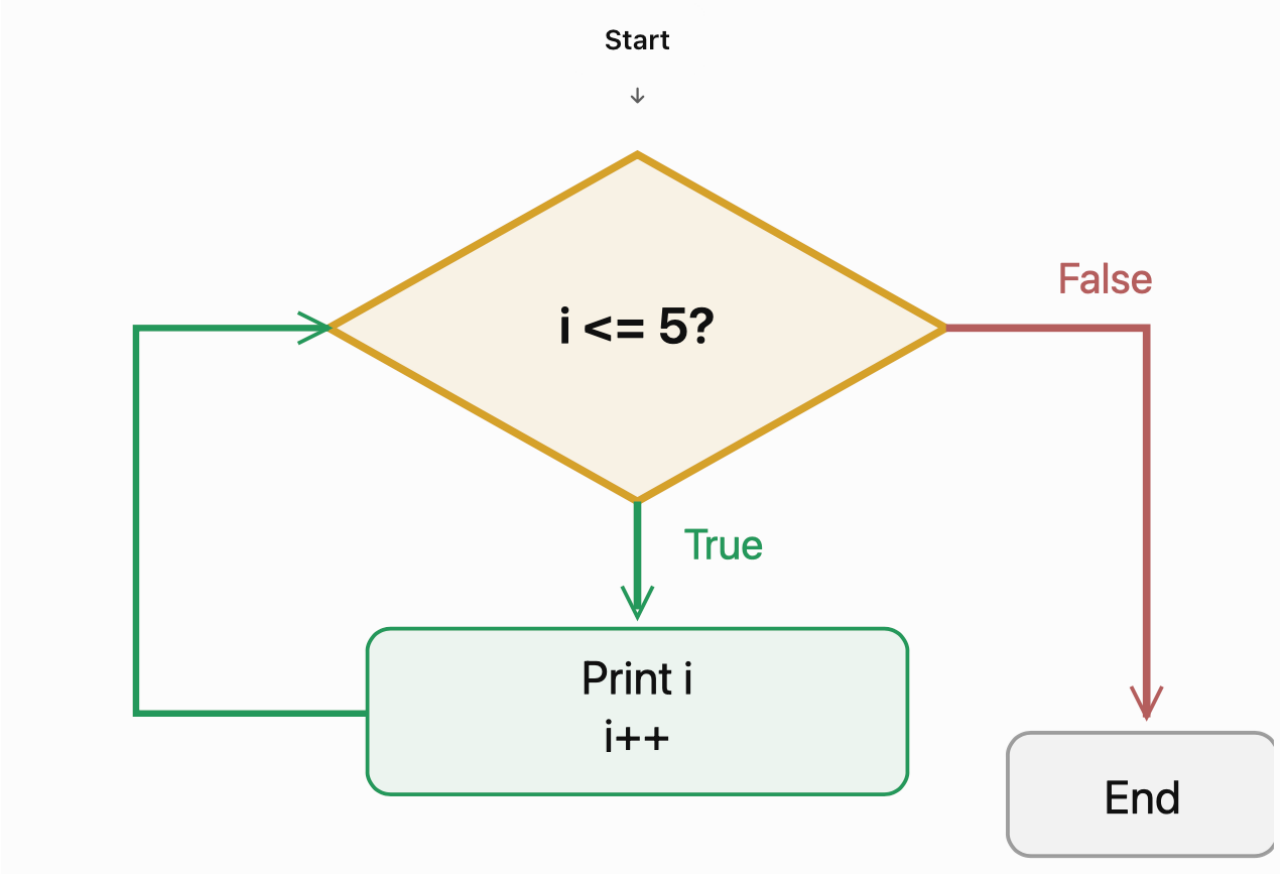
When `i = 6`, the condition `i <= 5` becomes false, and the loop stops.

JavaScript while Loop

What Is a while Loop?

The while loop executes a block of code repeatedly as long as a specified condition is true.

- The condition is checked before each iteration.
- If the condition is initially false, the loop never executes.
- Commonly used when the number of iterations is not known in advance.




```
while (condition) {
    // Code to execute
}

let i = 1;

while (i <= 5) {
    console.log(i);
    i++;
}
```

JavaScript do...while Loop

What Is a do...while Loop?

The do...while loop executes a block of code at least once, then repeats it as long as the condition remains true.

- The code block executes before checking the condition.
- The condition is evaluated after each iteration.
- Useful when an operation must run at least once.

```
do {
    // Code to execute
} while (condition);

let i = 1;

do {
    console.log(i);
    i++;
} while (i <= 5);
```

Key Difference: `while` vs. `do...while`

`while` Loop

JavaScript

```
let x = 10;

while (x < 5) {
    console.log(x);
    x++;
}
```

Output:

No output

The condition is false from the beginning.

`do...while` Loop

JavaScript

```
let x = 10;

do {
    console.log(x);
    x++;
} while (x < 5);
```

Output:

10

The code executes once before checking the condition.

Always ensure that the loop's condition can eventually become false. Otherwise, you may create an infinite loop.



Chapter 4
Functions

Amjad



Introduction to JavaScript Functions

What Is a Function?

A function is a reusable block of code designed to perform a specific task.

- Functions help organize code into smaller, manageable blocks.
- A function executes when it is called (invoked).
- Functions can receive input values called parameters.
- Functions can return a result using the return statement.
- Functions reduce code duplication and improve readability.

Defining a Function

A function must be defined before it can perform its task. A function declaration uses the function keyword, followed by a name, parentheses (), and a block of code {}
.

Key Concepts

- Function Declaration: Defines what the function does.
- Function Call: Executes the function.
- Reusability: A function can be called multiple times without rewriting its code.
- Hoisting: Function declarations can generally be called before their declaration appears in the code.

```
// Defining a Function
function functionName() {
    // Statements
}

functionName(); // Call the function

// Ex:
function sayHello() {
    console.log("Welcome to JavaScript!");
}

// Calling a Function Multiple Times
sayHello();
sayHello();
sayHello();
```

Anjal

Function Parameters and Arguments

What Are Parameters and Arguments?

Functions can accept input values to perform operations on different data.

- **Parameters**: Variables listed in the function definition.
- **Arguments**: Actual values passed to the function when calling it.
- A function can accept zero, one, or multiple parameters.

```
// Syntax
function functionName(parameter1, parameter2) {
    // Use the parameters
}

functionName(argument1, argument2);

// Example
function greet(name) {
    console.log("Hello, " + name + "!");
}

greet("Ali");
greet("Sara");

function add(a, b) {
    console.log(a + b);
}

add(10, 5); // 15
add(20, 8); // 28
```

The return Statement

What Is the `return` Statement?

The return statement is used to send a value back to the code that called the function.

- Returns a result from a function.
- Immediately stops the function's execution.
- Allows the returned value to be stored in a variable or used in another expression.
- A function without an explicit returned value returns undefined.

```
// Syntax
function functionName() {
    return value;
}

// Example
function add(a, b) {
    return a + b;
}

let result = add(10, 5);
console.log(result);
```



What is javascript?

Amjad







Author

Amjad Ataallah

Email: amjad.atallah@najah.edu
Linkedin: https://www.linkedin.com/in/ataallah/
Website: https://amjadataallah.github.io

Yanal Sweseh

Email: yanal.swaseh@najah.edu
Linkedin: https://www.linkedin.com/in/yanalsweseh/
Website: https://github.com/yswesieh